

ЛАБОРАТОРНАЯ РАБОТА №0

Тема: Погрешности. Приближенные вычисления. Вычислительная устойчивость.

Цель работы: Изучить особенности организации вычислительных процессов, связанные с погрешностями, приближенным характером вычислений на компьютерах современного типа, вычислительной устойчивостью.

Цель работы обуславливает постановку и решение следующих задач:

- 1) Рассмотреть источники погрешности в решении численных задач и способы их оценки.
- 2) Изучить особенности работы с машинными числами как с результатом дискретной проекции вещественных чисел на конкретную архитектуру компьютера.
- 3) Выяснить условия обеспечения вычислительной устойчивости решения численных задач.
- 4) Выполнить индивидуальное задание, закрепляющее на практике полученные знания (номер задания соответствует номеру студента по журналу; если этот номер больше, чем максимальное число заданий, тогда вариант задания вычисляется по формуле: номер по журналу % максимальный номер задания, где % - остаток от деления). На основании представленных вычислительных схем («прямой» и улучшенной) подобрать такие входные данные, что в первом случае схема демонстрировала бы заметную потерю в точности, а вторая на тех же входных данных — улучшала бы результат. В ходе выполнения данного задания следует использовать любой нескриптовый язык программирования, поддерживающий работу с машинными числами одинарной точности. Скриптовый язык Python, использованный для описания вычислительной схемы в задании, не использовать.
- 5) Отразить в отчете все полученные результаты. Сделать выводы.

Задание №1 Запустить и проинтерпретировать результаты работы разных вычислительных схем для

простого арифметического выражения на языке Rust. Код программы:

```
fn main() {  
    let num1: f32 = 0.23456789;  
    let num2: f32 = 1.5678e+20;  
    let num3: f32 = 1.2345e+10;  
  
    let result1 = (num1 * num2) / num3;  
    let result2 = (num1 / num3) * num2;  
    let result3: f64 = num1 as f64 * num2 as f64 / num3 as f64;  
  
    println!("({} * {}) / {} = {}", num1, num2, num3, result1);  
    println!("({} / {}) * {} = {}", num1, num3, num2, result2);  
    println!("{}", num1 * num2 / num3, result3);  
}
```

Результат работы программы:

```
Finished dev [unoptimized + debuginfo] target(s) in 0.02s  
Running `target\debug\lab0VM.exe`  
(0.2345679 * 15678000000000000000) / 12345000000 = 2978983700  
(0.2345679 / 12345000000) * 15678000000000000000 = 2978984000  
0.2345679 * 15678000000000000000 / 12345000000 = 2978983717.267449  
  
Process finished with exit code 0
```

Вывод: при обычных операциях над большими числами происходит заметная потеря в точности

(например решение 1 примера, где ответ = 2 978 983 828,43, выдаёт число 2 978 983 700, видно, что довольно большая разница между ними)

Но, если привести все переменные к числу с плавающей точкой двойной точности, в результате мы увидим реальный ответ, полностью совпадающий с нужным

Задание №2 Запустить и проинтерпретировать результаты работы разных вычислительных схем для итерационного и неитерационного вычисления на языке Rust.

Код программы:

```
fn main() {
    let numbers = [1.0f32, 20., 300., 4000., 5e6, f32::MIN_POSITIVE,
        f32::MAX*0.99]; // вектор с числами одинарной точности
    let iterations = 10; // число итераций

    for &number in &numbers {
        let mut result = number;

        for _ in 0..iterations {
            result = result.sqrt(); // послед. извлечение квадратного корня
        }

        for _ in 0..iterations {
            result = result * result; // послед. возведение числа в квадрат
        }

        let error = (number - result).abs();

        println!("Исх-е значение: {:e}, результат: {:e}, абс-ая погрешность:
{:e}, отн-ая погрешность: {:e} (%)", number, result, error, error * 100. /
number);
    }
}
```

Результат работы программы:

```
Finished dev [unoptimized + debuginfo] target(s) in 0.02s
Running `target\debug\lab0VM.exe`
Исх-е значение: 1e0, результат: 1e0, абс-ая погрешность: 0e0, отн-ая погрешность: 0e0 (%)
Исх-е значение: 2e1, результат: 2.000009e1, абс-ая погрешность: 8.9645386e-5, отн-ая погрешность: 4.4822693e-4 (%)
Исх-е значение: 3e2, результат: 3.0001422e2, абс-ая погрешность: 1.4221191e-2, отн-ая погрешность: 4.740397e-3 (%)
Исх-е значение: 4e3, результат: 4.0001064e3, абс-ая погрешность: 1.0644531e-1, отн-ая погрешность: 2.6611327e-3 (%)
Исх-е значение: 5e6, результат: 4.9994865e6, абс-ая погрешность: 5.135e2, отн-ая погрешность: 1.027e-2 (%)
Исх-е значение: 1.1754944e-38, результат: 1.17548e-38, абс-ая погрешность: 1.43e-43, отн-ая погрешность: 1.2159348e-3 (%)
Исх-е значение: 3.3687953e38, результат: 3.3686973e38, абс-ая погрешность: 9.796404e33, отн-ая погрешность: 2.9079844e-3 (%)

Process finished with exit code 0
```

Вывод: Чем больше значение числа, тем больше погрешность в итоге будет

Задание №3 С помощью программы на языке Rust вывести на экран двоичное представление машинных чисел одинарной точности стандарта IEEE 754 для записи: числа π , бесконечности, нечисла (NaN), наименьшего положительного числа, наибольшего положительного числа, наименьшего отрицательного числа. Сформулировать обоснование полученных результатов в пунктах 1 и 2, опираясь на двоичное представление машинных чисел.

Код программы:

```
fn main() {
    let pi = std::f32::consts::PI;
    let infinity = std::f32::INFINITY;
    let nan = std::f32::NaN;
    let smallest_positive = std::f32::MIN_POSITIVE;
    let largest_positive = std::f32::MAX;
    let smallest_negative = -std::f32::MIN_POSITIVE;

    println!(" $\pi$ : {}", float_to_binary_string(pi));
    println!("Infinity: {}", float_to_binary_string(infinity));
    println!("NaN: {}", float_to_binary_string(nan));
    println!("Smallest Positive Number: {}",
             float_to_binary_string(smallest_positive));
    println!("Largest Positive Number: {}",
             float_to_binary_string(largest_positive));
    println!("Smallest Negative Number: {}",
             float_to_binary_string(smallest_negative));
}

fn float_to_binary_string(num: f32) -> String {
    let bits = num.to_bits();
    format!("{:032b}", bits)
}
```

Результат работы программы:

```
Finished dev [unoptimized + debuginfo] target(s) in 0.07s
Running `target\debug\lab0VM.exe`
 $\pi$ : 0100000001001001000011111011011
Infinity: 01111111100000000000000000000000
NaN: 01111111100000000000000000000000
Smallest Positive Number: 00000000100000000000000000000000
Largest Positive Number: 01111110111111111111111111111111
Smallest Negative Number: 10000000100000000000000000000000

Process finished with exit code 0
```

Можно сделать вывод, что чем больше единиц в числе, тем больше погрешность при вычислении будет в итоге

4) Выполнить индивидуальное задание, закрепляющее на практике полученные знания

7.	$res = a[0]*x[0] + a[1]*x[1] + \dots + a[n-1]*x[n-1]$	<pre>prepare = sorted([(abs(a), x) for a, x in zip(a_values, x_values)], reverse=True) res = sum(a*x for a, x in prepare)</pre>
----	---	---

```
use std::io;

fn main() {
    let a= [100000.0, -0.0001, 1000.0]; // Пример значений для a
    let x= [0.00001, -1000.0, 1000000.0]; // Пример значений для x
    let result = a[0] * x[0] + a[1] * x[1] + a[2] * x[2];

    let a_values: Vec<f64> = vec![100000.0, -0.0001, 1000.0]; // Пример
значений для a
    let x_values: Vec<f64> = vec![0.00001, -1000.0, 1000000.0]; // Пример
значений для x

    println!("Результат: {}", result);

    let mut prepare: Vec<(f64, f64)> =
a_values.iter().zip(x_values.iter()).map(|(&a, &x)| (a.abs(), x)).collect();
    prepare.sort_by(|&(a1, _), &(a2, _)| a2.partial_cmp(&a1).unwrap());

    let res: f64 = prepare.iter().map(|&(a, x)| a * x).sum();

    println!("Результат: {}", res);
}
```

```
Finished dev [unoptimized + debuginfo] target(s) in 0.01s
Running `target\debug\lab0VM.exe`
Результат: 1000000001.1
Результат: 1000000000.9

Process finished with exit code 0
```

Контрольные вопросы

1. Как вы понимаете, что такое погрешность в контексте численных вычислений и почему с ней важно уметь работать и оценивать?
2. Каковы основные источники погрешности в численных задачах?
3. Какую погрешность называют неустранимой?
4. Что такое погрешность метода?
5. Что такое вычислительная погрешность?
6. Что такое абсолютная и относительная погрешности?
7. Что такое приближенные числа?
8. Что такое округление? Методы округления.
9. Что такое машинное число и как оно связано с вычислительной погрешностью?
10. Каковы меры обеспечения вычислительной устойчивости в численных задачах?
11. Приведите примеры чувствительности результата вычисления к последовательности математических операций.
12. Как избежать накопления погрешности в итерационном процессе?
13. Как алгоритм Кэхэна помогает уменьшить погрешность при суммировании чисел?
14. Как попарное суммирование помогает уменьшить погрешность?